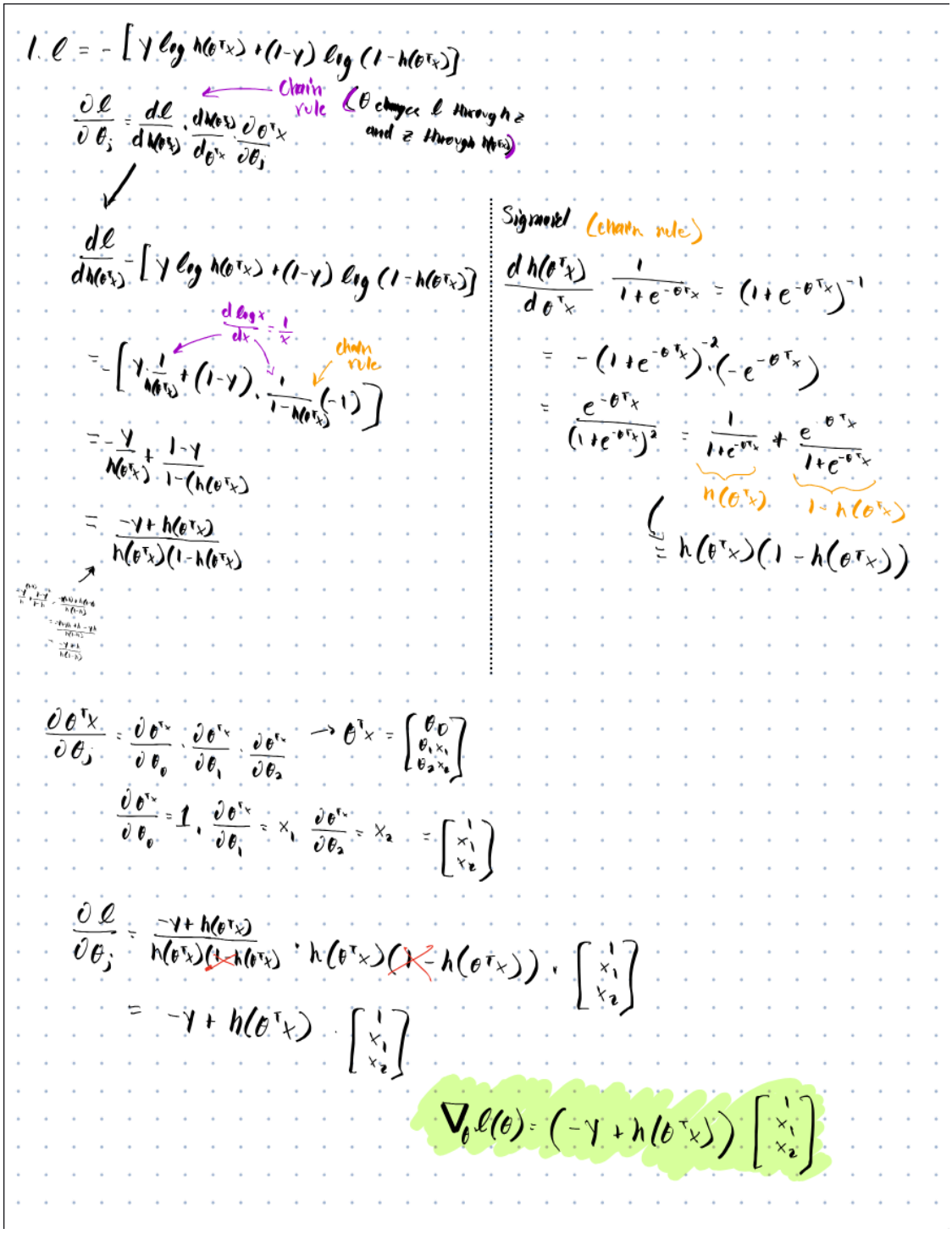
Simon Schmidt
10.18.26
1235114116
sschmi61

Part 1

1. Derivation of loss gradient



Part 2

2.1 Define the model

```python

class Model(nn.Module):
    def __init__(self, device="cpu"):
        super().__init__()
        self.linear_layer = nn.Linear(2, 1)

    def forward(self, x):
        # [TODO] your code
        lin_score = self.linear_layer(x)
        probabilty = torch.sigmoid(lin_score)
        return probabilty


trainloader = data.DataLoader(
    data.TensorDataset(torch.Tensor(1), torch.Tensor(1)), batch_size=1, shuffle=True
)
device = torch.device("cpu")

seed = 0
np.random.seed(seed)
torch.random.manual_seed(seed)
random.seed(seed)

X, y = sample_gaussian()
fig, ax = plt.subplots(1, 1, figsize=(5, 5))
plot_scatter(ax, X, y)

model = Model().to(device)
optimizer = optim.AdamW(model.parameters(), lr=1e-2, weight_decay=1e-6)

```

2.2 Train with automatic differentiation

```python

criterion = nn.BCELoss()
for itr in range(1, 2001):
    optimizer.zero_grad()
    yh = model(X)  # forward pass
    # compute loss # [TODO] your code
    loss = criterion(yh.squeeze(-1), y.float())
    print(itr, loss.item())
    loss.backward()  # backward pass
    optimizer.step()

```

last 40 iterations:

```
1960 0.007607827428728342
1961 0.007601830642670393
1962 0.007595841772854328
1963 0.007589858490973711
1964 0.007583882194012403 5
1965 0.007577913347631693
1966 0.007571953348815441
1967 0.00756599847227335
1968 0.007560050114989281
1969 0.007554108276963234
1970 0.007548174820840359
1971 0.007542248349636793
1972 0.007536329329013824 5
1973 0.007530414499342441 6
1974 0.007524508517235517 5
1975 0.00751861184835434
1976 0.007512719370424747 5
1977 0.00750683108344674 1
1978 0.00750095397233963
1979 0.007495081983506679 5
1980 0.00748921791091561 3
1981 0.007483361288905144
1982 0.00747750885784626
1983 0.007471665274351835
1984 0.007465827278792858
1985 0.007459997199475765
1986 0.007454172242432833
1987 0.007448355667293072
1988 0.007442544214427471
1989 0.007436741143465042
1990 0.007430944591760635
1991 0.00742515455931425 1
1992 0.007419370580464601 5
1993 0.00741359498351812 4
1994 0.007407825440168381
1995 0.007402059156447649
1996 0.007396304979920387
1997 0.00739055452868342 4
1998 0.00738481199368834 5
1999 0.00737907737493515
2000 0.007373346015810966 5
```

2.3 Plot predictions and the decision boundary

```
with torch.no_grad():
    fig = plt.figure(figsize=(4, 4))
    ax = plt.gca()

    probability = model(X).squeeze(-1)
    predicted = (probability >= 0.5).long()
    plot_scatter(ax, X, predicted)

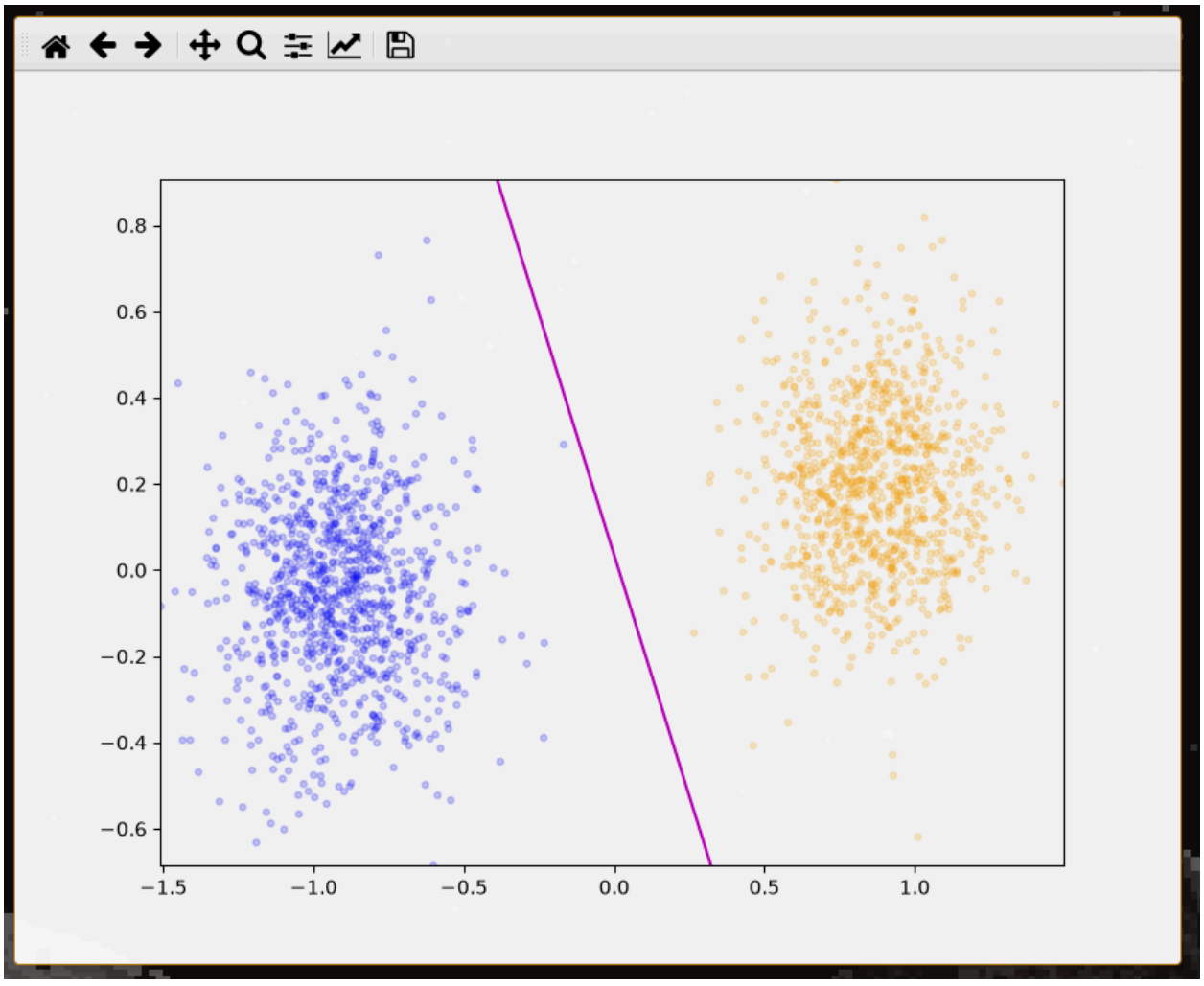
    theta0 = model.linear_layer.bias[0].item()
    theta1 = model.linear_layer.weight[0, 0].item()
    theta2 = model.linear_layer.weight[0, 1].item()

    x1_min = X[:, 0].min().item()
    x1_max = X[:, 0].max().item()
    x2_min = X[:, 1].min().item()
    x2_max = X[:, 1].max().item()

    if abs(theta2) > 1e-8:
        x1_val = np.linspace(x1_min, x1_max, 100)
        x2_val = -(theta0 + theta1 * x1_val) / theta2
        ax.plot(x1_val, x2_val, "m")

    else:
        ax.axvline(-theta0 / theta1, color="m")

    ax.set_xlim(x1_min, x1_max)
    ax.set_ylim(x2_min, x2_max)
    plt.show()
```



Part 3

3.1 Implement binary cross-entropy

```python
model2 = Model().to(device)

# print(model2.linear_layer.weight)
# print(model2.linear_layer.bias)


def binary_cross_entropy(y_true, y_pred, epsilon=1e-12):
    """
    Compute the binary cross-entropy loss.

    Parameters:
    y_true (torch array): Ground truth labels (0 or 1).
    y_pred (torch array): Predicted probabilities (between 0 and 1).
    epsilon (float): Small value to prevent log(0). [NOTE] do not change this

    Returns:
    float: Binary cross-entropy loss.
    """
    # Clip predictions to avoid log(0) issues
    y_pred = torch.clip(y_pred, epsilon, 1 - epsilon)

    # Compute BCE loss
    loss = -torch.mean(
        y_true * torch.log(y_pred) + (1 - y_true) * torch.log(1 - y_pred)
    )
    return loss

```

3.2 Implement the manual training loop

```python
learning_rate = 1e-2
for t in range(2000):
    # Forward pass: compute predicted y
    yh = model2(X)

    # Compute and print loss
    loss = binary_cross_entropy(
        y.float(), yh.squeeze(-1)
    )  # np.square(y_pred - y).sum()
    print(loss)

    # [TODO] your code: iteratively update the model parameters
    # The model parameters theta_0, theta_1, theta_2 can be updated individually
    # by (1) computing the gradient using the analytical expression you obtained from the part 1 of the homework
    # by (2) updating the model parameters into the "DESCENDING" direction by the step size (that is, learning rate)
    with torch.no_grad():
        l_theta = -y.float() + yh.squeeze(-1)
        grad_theta_0 = l_theta.mean()
        grad_theta_1 = (l_theta * X[:, 0]).mean()
        grad_theta_2 = (l_theta * X[:, 1]).mean()

        model2.linear_layer.bias[0] -= learning_rate * grad_theta_0
        model2.linear_layer.weight[0, 0] -= learning_rate * grad_theta_1
        model2.linear_layer.weight[0, 1] -= learning_rate * grad_theta_2
```

last 40 iterations:

```
tensor(0.0779, grad_fn=<NegBackward0>)
tensor(0.0778, grad_fn=<NegBackward0>)
tensor(0.0778, grad_fn=<NegBackward0>)
tensor(0.0778, grad_fn=<NegBackward0>)
tensor(0.0777, grad_fn=<NegBackward0>)
tensor(0.0777, grad_fn=<NegBackward0>)
tensor(0.0777, grad_fn=<NegBackward0>)
tensor(0.0776, grad_fn=<NegBackward0>)
tensor(0.0776, grad_fn=<NegBackward0>)
tensor(0.0776, grad_fn=<NegBackward0>)
tensor(0.0775, grad_fn=<NegBackward0>)
tensor(0.0775, grad_fn=<NegBackward0>)
tensor(0.0775, grad_fn=<NegBackward0>)
tensor(0.0774, grad_fn=<NegBackward0>)
tensor(0.0774, grad_fn=<NegBackward0>)
tensor(0.0774, grad_fn=<NegBackward0>)
tensor(0.0773, grad_fn=<NegBackward0>)
tensor(0.0773, grad_fn=<NegBackward0>)
tensor(0.0773, grad_fn=<NegBackward0>)
tensor(0.0772, grad_fn=<NegBackward0>)
tensor(0.0772, grad_fn=<NegBackward0>)
tensor(0.0772, grad_fn=<NegBackward0>)
tensor(0.0771, grad_fn=<NegBackward0>)
tensor(0.0771, grad_fn=<NegBackward0>)
tensor(0.0771, grad_fn=<NegBackward0>)
tensor(0.0770, grad_fn=<NegBackward0>)
tensor(0.0770, grad_fn=<NegBackward0>)
tensor(0.0770, grad_fn=<NegBackward0>)
tensor(0.0769, grad_fn=<NegBackward0>)
tensor(0.0769, grad_fn=<NegBackward0>)
tensor(0.0769, grad_fn=<NegBackward0>)
tensor(0.0768, grad_fn=<NegBackward0>)
tensor(0.0768, grad_fn=<NegBackward0>)
tensor(0.0768, grad_fn=<NegBackward0>)
tensor(0.0767, grad_fn=<NegBackward0>)
~/ASU/fall_26/cse_475/475_assignments/hw2 git master ?1 ❮                    4m 27s 22:43:35
```

3.3 Plot predictions and the decision boundary

```python
with torch.no_grad():
    fig = plt.figure(figsize=(4, 4))
    ax = plt.gca()

    probability = model2(X).squeeze(-1)
    predicted = (probability >= 0.5).long()
    plot_scatter(ax, X, predicted)

    theta0 = model2.linear_layer.bias[0].item()
    theta1 = model2.linear_layer.weight[0, 0].item()
    theta2 = model2.linear_layer.weight[0, 1].item()

    x1_min = X[:, 0].min().item()
    x1_max = X[:, 0].max().item()
    x2_min = X[:, 1].min().item()
    x2_max = X[:, 1].max().item()

    if abs(theta2) > 1e-8:
        x1_val = np.linspace(x1_min, x1_max, 100)
        x2_val = -(theta0 + theta1 * x1_val) / theta2
        ax.plot(x1_val, x2_val, "m")

    else:
        ax.axvline(-theta0 / theta1, color="m")

    ax.set_xlim(x1_min, x1_max)
    ax.set_ylim(x2_min, x2_max)
    plt.show()
```

